

Teaching Google Antigravity to Paint: A Stateful Image-Editing Skill Built on Gemini's Interactions API and MCP

TL;DR: [nb2lite-skill-agy](#) wraps Google's `gemini-3.1-flash-lite-image` model (NB2Lite) in a FastMCP server and packages it as an Antigravity CLI skill. You type "generate an image of a cyberpunk kitchen" into Antigravity, and it just... does it. Then you say "add a neon RAMEN sign" and it edits *the same image* without re-prompting the whole scene. Oh, and the cover image of this article? Generated by the thing the article is about — dogfooding all the way down. More on that at the end.

Background: why another image tool?

Most image-generation workflows are **stateless**. You send a prompt, you get pixels back, and the model immediately forgets everything. Want to tweak the result? You re-describe the *entire scene* and pray the character, lighting, and composition survive the round trip. (Narrator: they don't.)

Google's **NB2Lite** — the friendly nickname for `gemini-3.1-flash-lite-image` — takes a different approach. It's a high-efficiency image model with sub-2-second generations, solid text rendering in 25+ languages, and — the headline feature — support for the **stateful Interactions API**, which lets you iterate on an image across multiple turns while the model keeps the visual context server-side.

This repo glues that capability directly into **Google Antigravity CLI**, so your coding agent can generate and iteratively refine images as a natural part of a pair-programming session. It ships as two things in one repo:

1. A **Model Context Protocol (MCP) server** (`nb2lite-agent`, a single-file FastMCP app in `server.py`) exposing four tools.
2. A **Skill definition** (`nb2lite-image`) that teaches Antigravity *when* and *how* to use those tools well.

The Interactions API: images with a memory

The Interactions API is Gemini's stateful endpoint. The core loop looks like this:

1. You call `client.interactions.create(...)` with a prompt and `store=True`.
2. The response includes an **interaction_id** — a handle to the turn's visual context, persisted on Google's servers.
3. On the next call, you pass `previous_interaction_id`, and the model edits the *existing canvas* — preserving character, style, lighting, and pixel continuity.

So instead of this (stateless suffering):

"A watercolor fox in a forest at dawn, mist, soft light, wearing a red scarf, three birch trees on the left, *and now also* holding a lantern"

...you write this in Antigravity:

"Add a lantern in its paw."

That's it. The stored context holds the rest.

A few practical details the server handles for you:

- **Every turn returns a *new* interaction ID.** Chain the latest one; editing from a stale ID silently forks your session from an older state.
- **Aspect ratio is chosen at generation time** (1:1, 16:9, 9:16, 4:3, 3:4) and *inherited* on stateful edits — changing it mid-session degrades pixel continuity, so the edit tool deliberately doesn't accept one.
- **Thinking levels:** low (default, fast drafts) or high (complex rendering, accurate text layout, character composition). The generic API spec also lists minimal and medium, but the live API rejects them for this model with an HTTP 400 — the server saves you from discovering that the hard way.

What is MCP, in one minute

The **Model Context Protocol** is an open standard for connecting AI assistants to tools and data. Before it, giving a model access to some service meant writing a bespoke integration for each assistant — N assistants × M services, everyone reinventing the same plumbing. MCP collapses that: a tool author writes one **MCP server** that exposes typed tools, and any MCP-capable client like Antigravity CLI can discover and call them with no per-client glue code.

An MCP server is usually a small local process that speaks JSON-RPC over stdio. Antigravity launches it, asks "what tools do you have?", and from then on the model can call them like native functions.

The nb2lite-agent server exposes four core tools:

Tool	What it does
generate_image	Text → 1k image. Saves locally, returns the path + an interaction ID.
edit_image	Stateful edit: takes the previous interaction ID + a description of <i>only the change</i> .
edit_local_image	Uploads any local image file inline (base64) and applies an edit — your entry point for existing files.
get_help	Reports live config: API key status, active model, output directory, full tool reference.

Images land on disk as `gen_<timestamp>_<uuid8>.jpg` (or `edit_/edit_local_` prefixed) — the UUID suffix keeps concurrent generations from clobbering each other. Errors come back as 🚫 . . . text strings rather than protocol errors, so Antigravity can read and react to them gracefully.

And what's an Agent *skill*?

If MCP is the *hands* (the tools an agent can physically call), a **skill** is the *muscle memory* — a markdown file (`SKILL.md`) plus bundled resources that load into Antigravity's context and teach it the workflow: which tool to reach for, in what order, with which constraints.

For nb2lite-image, the skill encodes things like:

- Call `get_help` first when diagnosing setup issues — if the API key is missing, nothing else will work.
- Keep edit prompts **incremental**: describe the change, not the scene.
- Always chain the **latest** interaction ID.
- Generations are billable — batch related edits and prefer `thinking_level: low` for drafts.

The skill also bundles the MCP server itself (`mcp/server.py`), its requirements, an installer script, and a vendored copy of the Interactions API developer guide — so it's fully self-contained.

Installing it into Antigravity CLI

You need three things: **Python 3.10+**, **Antigravity CLI**, and a **Gemini API key** (free from [Google AI Studio](#)). Pick *one* of the paths below.

Path A: The plugin marketplace (fewest keystrokes)

Inside your Antigravity session, run:

```
/plugin marketplace add xbill9/nb2lite-skill-agy
/plugin install nb2lite-image@nb2lite-skill-agy
```

🔧

This installs the skill **and** auto-registers the MCP server. The plugin manifest carries no API key — the server reads `GEMINI_API_KEY` from your environment, so make sure it's exported before launching Antigravity CLI.

Path B: Clone and bootstrap (this repo)

```
# 1. Get the code
git clone https://github.com/xbill9/nb2lite-skill-agy.git
cd nb2lite-skill-agy

# 2. One-command setup: installs deps, registers the MCP server
#   in .mcp.json, and prompts for your API key (stored in ~/gemini.key)
./init.sh

# 3. Restart Antigravity CLI in this directory and approve the server
#   when prompted. Verify with:
/mcp          # should list nb2lite-agent
```

🔧

`init.sh` is idempotent and safe to rerun anytime.

Path C: Install into *your* project

From a clone of the repo:

```
make init TARGET=/path/to/your/project ARGS='--output-dir ./images'
```

🔧

This copies the skill into <project>/.*gemini*/antigravity-cli/skills/nb2lite-image/ and writes the nb2lite-agent entry into that project's *.mcp.json*. It reuses *~/gemini.key* if available. Restart Antigravity in your project, approve the server, done.

Path D: Docker (nothing on the host but Docker)

The server is published as [xbill9/nb2lite-agent](https://xbill9.github.io/nb2lite-agent/):

```
antigravity mcp add nb2lite-agent --env GEMINI_API_KEY="$(cat ~/gemini.key)" -- \
  docker run --rm -i -e GEMINI_API_KEY -v "$PWD:$PWD" -w "$PWD" xbill9/nb2lite-agent
```

🔧

The `-v "$PWD:$PWD" -w "$PWD"` mount ensures the container can save images to your workspace disk and read local files for `edit_local_image`.

Troubleshooting

- /mcp doesn't list the server → restart Antigravity CLI in the project directory.
- Tools return  `GEMINI_API_KEY is not set` → run `source set_env.sh` (or export the key) and restart.
- Anything else → ask Antigravity to call `get_help`; it reports the live configuration.

Examples: an Antigravity session in practice

Once installed, you talk to Antigravity in plain English. A real flow looks like:

You: *"Generate a cozy cabin in a snowy forest at dusk, 16:9."*

Antigravity calls:

```
generate_image(
  prompt="A cozy log cabin in a snowy forest at dusk, warm light in the windows",
  aspect_ratio="16:9",
  thinking_level="low",
)
#  Saved to: ./gen_1784759001_a1b2c3d4.jpg
# Interaction ID: v1_ChdpRU5...
```

🔧

You: *"Nice. Add smoke curling from the chimney."*

```
edit_image(
  previous_interaction_id="v1_ChdpRU5...",
  edit_prompt="add gentle smoke curling from the chimney",
)
#  Saved to: ./edit_1784759050_e5f6a7b8.jpg
# Interaction ID: v1_Xk9mPq2... ← a NEW id; the next edit chains this one
```

🔧

You: *"Now make it night, with aurora in the sky."*

Same tool, newest ID, and the cabin, trees, and chimney smoke all stay put — only the sky changes. No re-prompting, no continuity roulette.

And for images that didn't come from the model at all:

You: *"Take ./whiteboard-sketch.png and render it as a clean 3D product mockup."*

```
edit_local_image(  
    image_path="./whiteboard-sketch.png",  
    edit_prompt="render this hand-drawn sketch as a high-fidelity 3D product mockup",  
    aspect_ratio="4:3",  
)
```

⌘ ⇧ ⌘

It returns an interaction ID too — so follow-up refinements switch to `edit_image` and go stateful from there.

Dogfooding: about that cover image 🐶🍲

"Eating your own dog food" means using your own product for real work. It's the difference between "this should work" and "I ship with this every day."

This repo dogfoods itself at every layer:

- The **skill is active inside its own repository** — open Google Antigravity in a clone and the `nb2lite-image` skill and `nb2lite-agent` server are already wired up, so every development session doubles as an integration test.
- The **integration tests** (`make test`) drive the same four MCP tools an end user would, against the live API.
- And **the cover image of this article was generated by the exact skill the article describes**, from inside an Antigravity CLI session in this repo. One tool call, live generation, no retouching:

```
generate_image(  
    prompt="A wide tech blog cover illustration: a friendly AI agent with glowing antigravity elements floating al  
    aspect_ratio="16:9",  
    thinking_level="high",  
)  
# 🟢 Image successfully saved!  
# • Saved to: /home/xbill/nb2lite-skill-agy/gen_1784832091_d71439ca.jpg  
# • Interaction ID: v1_ChdxUjYpYXB5aEZZYkotOF1QeC1UcG1BNBIXV21CaWFweWhGWWJKLThZUHgtVHBtQTQ
```

⌘ ⇧ ⌘

(That exact output is committed to the repo as [devto-cover.jpg](#), receipts and all.)

Worth noticing:

- **The text rendered correctly.** "NB2Lite + Antigravity" came out crisp and typo-free — that's what `thinking_level: "high"` buys you on text-heavy layouts.
- **The model illustrated its own pitch.** The chain of frames (Initialize → Nebula Base → Enhance Detail → Refine → Stateful Edit) *is* the stateful edit loop — the image explains the Interactions API better than a manual diagram.
- **If I wanted the accent color changed**, I wouldn't regenerate — I'd `edit_image` with that interaction ID and say "make the cyan accents emerald." That's the whole point.

Dogfooding is the cheapest credibility there is: the tool's real output is literally the first thing you saw when you opened this article.

Links

- Repo: github.com/xbill9/nb2lite-skill-agy (Apache-2.0)
- Docker image: hub.docker.com/r/xbill9/nb2lite-agent
- Interactions API reference: ai.google.dev/api/interactions-api
- Model Context Protocol: modelcontextprotocol.io

This is a third-party community project, not affiliated with or endorsed by Google. Bring your own Gemini API key — and remember generations are billable, so draft on low and save high for final outputs.